

1.1 Die Queue / Schlange

Wiederholung Arrays / Felder

Felder (engl. Arrays) dienen der Speicherung einer festen Anzahl von Objekten gleichen Datentyps, welche über einen Index (Positionsanzahl) adressiert werden können. Beispiel Array vom Typ `String` der Größe 4:

Index i	textFeld[i]
0	"Hallo "
1	"Welt, "
2	"ein "
3	"Satz."

Aufgabe 1

- 
- a) Identifizieren Sie die Arrays im Projekt 1 `Queue`. Beachte speziell deren Erstellung, Initialisierung und Nutzung (lesend / schreibend)
- 
- b) Erstellen Sie eine neue Klasse `Test` und erstelle darin ein beliebiges Array. Belege zufällig einige (nicht alle) Felder mit Werten.
- c) Nutzen Sie eine Wiederholung, um alle noch leeren Felder mit einem Wert zu belegen.

Aufgabe 2

Verändern Sie das Programm so, dass eine zufällige Anzahl an Objekten gehalten werden kann.

Research: `java Math.random() range`

Probleme mit Arrays

Vervollständigen Sie den Lückentext mit den passenden Wörtern:

initialisiert – nicht initialisiert – ein neues Array – Speicher – seinen Nachfolger –
setzen – allokiert – nicht einfach erweitert

Die Belegung des **Arrays** ist recht umständlich. Das **Feld** muss zunächst erstellt und

initialisiert werden um anschließend die einzelnen **Elemente** an die richtige Stelle
setzen zu können. Auch kann das Array
nicht einfach erweitert werden, sondern es muss
ein neues Array **erstellt** und alle Elemente kopiert werden.

Zusätzlich verbraucht das Array sehr viel Speicher, auch wenn die
Objekte noch nicht initialisiert sind, da bereits zum Zeitpunkt der Erstellung des
Arrays entsprechend **RAM** allokiert wird.

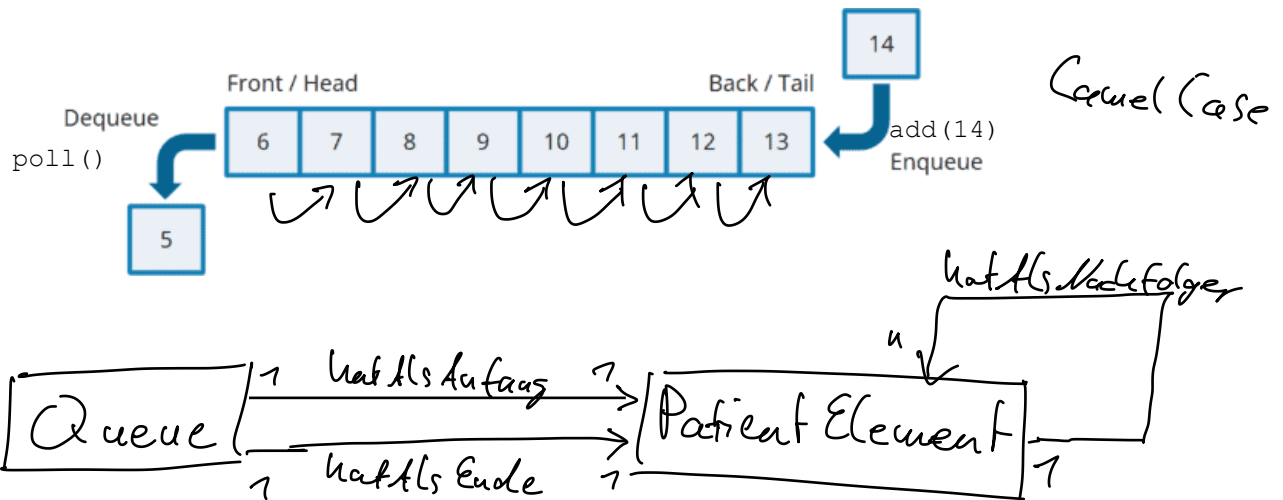
Um diese Probleme zu beheben, gibt es eine Reihe von **Datenstrukturen**. (In Java sind diese als Subinterfaces von `java.util.Collection` zusammengefasst)

Im Folgenden wird zunächst die Datenstruktur **Queue / Schlange** betrachtet.

Die Idee ist simpel: Jedes Element der Schlange kennt wie bisher seine eigenen Daten und nun zusätzlich noch seinen Nachfolger.

Aufgabe 3a) Ergänzen Sie die folgende Darstellung:

- für jedes Element um einen Verweis auf seinen Nachfolger.
- Die Datenstruktur Queue als zusätzliche Klassenkarte, welche nur einen Verweis auf den Head hält, sowie die Methoden `poll()` und `add(Object e)`



Element

- b) Vervollständigen Sie das Klassendiagramm mit der Klasse `Patient`. (Tipp: B.S.7) Ergänzen Sie dabei auch notwendige Attribute und Methoden wie im Projekt 1 Queue.

Merke:

Die Datenstruktur **Queue / Schlange** ermöglicht es eine beliebige Anzahl an Objekten gleichen Typs in einer bestimmten Reihenfolge zu speichern. Dabei hält die Queue selbst einen **Verweis auf das erste Objekt** in der Schlange und jedes Objekt in der Queue einen **Verweis auf seinen Nachfolger**. Dies stellt eine 1:n Beziehung dar, wobei n beliebig ist.

Eine Beziehung zwischen Objekten der gleichen Klasse nennt man **rekursive Beziehung**. Die „hat Nachfolger“ Beziehung zwischen den einzelnen Patientenelementen stellt eine solche dar.

Zur Umsetzung in Programmcode benötigen wir hierzu ein **Referenzattribut**, in diesem Fall also ein Attribut mit gleichem Datentyp wie die Klasse selbst.

Beim **Einfügen** in eine Queue wird ein Objekt **hinten angefügt** (`add()`). Beim **Herausnehmen** aus der Queue wird **von vorne weggenommen** (`poll()`).
 → Dieses Prinzip nennt man **FIFO – First In First Out**.

Aufgabe 4a) Notieren Sie je die passende Klasse, Methode und deren Länge (Zeilenanzahl)

- Patient hinzufügen (PMS Array): _____
- Patient hinzufügen (PMS Queue): _____
- Patient drannehmen (PMS Array): _____
- Patient drannehmen (PMS Queue): _____
- Erklären Sie auf einem Blockblatt die Umsetzung der Methoden in der `PatientQueue`.

- b) Erstellen Sie selbst eine Queue samt Datenelement (zwei Klassen) für ein Schlange von Taxis.